

EXPERIMENT - 32

1. Problem Statement

Write a C program to perform the following menu-driven operations using separate functions:

- Get two variable length 2D arrays and prove that matrix addition is commutative.
- Get two variable length 2D arrays and prove that matrix multiplication is not commutative.
- Exit.

2. Algorithm

Main Program

Step 1: START

Step 2: Display the menu and read choice.

Step 3: Using a switch statement on choice:

 If choice is 1, call additionCommutative().

 If choice is 2, call multiplicationNotCommutative().

 If choice is 3, display "Exiting the program."

Step 4: Repeat from Step 2 while choice is not equal to 3.

Step 5: STOP

Function additionCommutative()

Step 1: Read rows and cols, and declare the variable length arrays a, b, ab and ba, all of size rows × cols.

Step 2: Read the elements of matrix A and then of matrix B.

Step 3: Repeat for i = 0 to rows – 1:

 Repeat for j = 0 to cols – 1: set $ab[i][j] = a[i][j] + b[i][j]$ and $ba[i][j] = b[i][j] + a[i][j]$, and if the two are not equal set same = 0.

Step 4: Display A + B and B + A.

Step 5: If same is 1, display that addition is commutative.

Step 6: Return to the calling function.

Function multiplicationNotCommutative()

Step 1: Read n, the order of the two square matrices, and declare the variable length arrays a, b, ab and ba, all of size n × n.

Step 2: Read the elements of matrix A and then of matrix B.

Step 3: Repeat for i = 0 to n – 1 and for j = 0 to n – 1:

 Set $ab[i][j] = 0$ and $ba[i][j] = 0$.

 Repeat for k = 0 to n – 1: add $a[i][k] \times b[k][j]$ to $ab[i][j]$, and add $b[i][k] \times a[k][j]$ to $ba[i][j]$. Then if the two are not equal, set same = 0.

Step 4: Display A × B and B × A.

Step 5: If same is 0, display that multiplication is not commutative.

Step 6: Return to the calling function.

3. Test Case Table

#	Operation	Input	Expected Output	Actual Output	Result
1	1. Addition	2×3 , A = 1 2 3 / 4 5 6, B = 7 8 9 / 10 11 12	Both A + B and B + A come out as 8 10 12 / 14 16 18	Both identical	PASS
2	1. Addition	the same pair	The message says addition is commutative	A + B = B + A	PASS
3	2. Multiplication	2×2 , A = 1 2 / 3 4, B = 5 6 / 7 8	A $\times$ B = 19 22 / 43 50	19 22 / 43 50	PASS
4	2. Multiplication	the same pair	B $\times$ A = 23 34 / 31 46, which is different	23 34 / 31 46	PASS
5	2. Multiplication	3×3 , A = 1 to 9, B = 9 down to 1	A $\times$ B = 30 24 18 / 84 69 54 / 138 114 90	Same as expected	PASS
6	2. Multiplication	the same 3×3 pair	B $\times$ A = 90 114 138 / 54 69 84 / 18 24 30	Different from A $\times$ B	PASS
7	3. Exit	choice = 3	Exiting the program, and the loop ends	Program ended	PASS

4. Source Code

commutative.c

```
/* Experiment 32. Addition is commutative, multiplication is not.
   Both options use variable length arrays, because the order of the
   matrices is only known at run time. */

#include <stdio.h>

void additionCommutative(void);
void multiplicationNotCommutative(void);

/* The number of columns comes before the matrix, so that the compiler
   knows the shape of each row. */
void readMatrix(int rows, int cols, int m[rows][cols], char name)
{
    int i, j;

    printf("Enter the elements of matrix %c (%d x %d):\n", name, rows, cols);
    for (i = 0; i < rows; i++)
        for (j = 0; j < cols; j++)
            scanf("%d", &m[i][j]);
}

void showMatrix(int rows, int cols, int m[rows][cols], const char *label)
{
    int i, j;

    printf("%s\n", label);
    for (i = 0; i < rows; i++) {
        for (j = 0; j < cols; j++) {
            if (j > 0)
                printf("\t");
            printf("%d", m[i][j]);
        }
        printf("\n");
    }
}

int main(void)
{
    int choice;

    do {
        printf("\nMENU\n");
        printf("1. Prove that matrix addition is commutative\n");
        printf("2. Prove that matrix multiplication is not commutative\n");
        printf("3. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1: additionCommutative(); break;
            case 2: multiplicationNotCommutative(); break;
            case 3: printf("Exiting the program.\n"); break;
        }
    } while (choice != 3);

    return 0;
}

/* Both sums are worked out separately and then compared place by place,
   instead of being assumed equal. */
void additionCommutative(void)
{
    int rows, cols, i, j, same = 1;

    printf("Enter the number of rows and columns: ");
    scanf("%d %d", &rows, &cols);
```

```

int a[rows][cols], b[rows][cols], ab[rows][cols], ba[rows][cols];

readMatrix(rows, cols, a, 'A');
readMatrix(rows, cols, b, 'B');

for (i = 0; i < rows; i++)
    for (j = 0; j < cols; j++) {
        ab[i][j] = a[i][j] + b[i][j];
        ba[i][j] = b[i][j] + a[i][j];
        if (ab[i][j] != ba[i][j])
            same = 0;
    }

showMatrix(rows, cols, ab, "A + B =");
showMatrix(rows, cols, ba, "B + A =");

if (same)
    printf("A + B = B + A, so matrix addition is commutative\n");
else
    printf("A + B is not equal to B + A\n");
}

/* Two square matrices of the same order are used, because A x B and
   B x A both have to exist and be the same size to be compared. */
void multiplicationNotCommutative(void)
{
    int n, i, j, k, same = 1;

    printf("Enter the order of the two square matrices: ");
    scanf("%d", &n);

    int a[n][n], b[n][n], ab[n][n], ba[n][n];

    readMatrix(n, n, a, 'A');
    readMatrix(n, n, b, 'B');

    for (i = 0; i < n; i++)
        for (j = 0; j < n; j++) {
            ab[i][j] = 0;
            ba[i][j] = 0;
            for (k = 0; k < n; k++) {
                ab[i][j] += a[i][k] * b[k][j];
                ba[i][j] += b[i][k] * a[k][j];
            }
            if (ab[i][j] != ba[i][j])
                same = 0;
        }

    showMatrix(n, n, ab, "A x B =");
    showMatrix(n, n, ba, "B x A =");

    if (!same)
        printf("A x B is not equal to B x A, so matrix multiplication is not commutative\n");
    else
        printf("A x B = B x A for this pair, but that does not hold for every pair\n");
}

```

5. Compilation and Output

I compiled it with `gcc -Wall -Wextra commutative.c -o commutative` so that even the small warnings would show up, and it compiled clean in the first attempt itself.

Addition of a 2×3 Pair

```
adilshakil — zsh — 120x33

Last login: Thu Sep 24 22:24:23 on ttys000
adilshakil@ADILs-MacBook-Air MyAssignment % gcc -Wall -Wextra commutative.c -o commutative
adilshakil@ADILs-MacBook-Air MyAssignment % ./commutative

MENU
1. Prove that matrix addition is commutative
2. Prove that matrix multiplication is not commutative
3. Exit
Enter your choice: 1
Enter the number of rows and columns: 2 3
Enter the elements of matrix A (2 x 3):
1 2 3
4 5 6
Enter the elements of matrix B (2 x 3):
7 8 9
10 11 12
A + B =
8      10      12
14     16      18
B + A =
8      10      12
14     16      18
A + B = B + A, so matrix addition is commutative
█
```

Multiplication of a 2×2 Pair

```
adilshakil — zsh — 120x33

MENU
1. Prove that matrix addition is commutative
2. Prove that matrix multiplication is not commutative
3. Exit
Enter your choice: 2
Enter the order of the two square matrices: 2
Enter the elements of matrix A (2 x 2):
1 2
3 4
Enter the elements of matrix B (2 x 2):
5 6
7 8
A x B =
19      22
43      50
B x A =
23      34
31      46
A x B is not equal to B x A, so matrix multiplication is not commutative
█
```

Multiplication of a 3×3 Pair, and Exit

```
adilshakil — zsh — 120x33

MENU
1. Prove that matrix addition is commutative
2. Prove that matrix multiplication is not commutative
3. Exit
Enter your choice: 2
Enter the order of the two square matrices: 3
Enter the elements of matrix A (3 x 3):
1 2 3
4 5 6
7 8 9
Enter the elements of matrix B (3 x 3):
9 8 7
6 5 4
3 2 1
A x B =
30      24      18
84      69      54
138     114     90
B x A =
90      114     138
54      69      84
18      24      30
A x B is not equal to B x A, so matrix multiplication is not commutative

MENU
1. Prove that matrix addition is commutative
2. Prove that matrix multiplication is not commutative
3. Exit
Enter your choice: 3
Exiting the program.
adilshakil@ADILs-MacBook-Air MyAssignment %
```

All seven test cases came out right, and I checked the 2×2 products by hand as well: $A \times B$ came to 19 22 / 43 50 and $B \times A$ to 23 34 / 31 46, which is what the program printed.

One thing I had to think about in option 2 is why both matrices are square and of the same order. For $A \times B$ to exist the columns of A must match the rows of B, and for $B \times A$ to exist it has to hold the other way round too. Square matrices of the same order are the simplest case where both products exist and are of the same size, so the two answers can actually be compared place by place. For addition no such condition is needed beyond both matrices being of the same order, because addition works place by place and $a + b$ is the same as $b + a$ for plain numbers.